



University of Bologna
Dipartimento di Informatica –
Scienza e Ingegneria (DISI)
Engineering Bologna Campus

Class of
**Infrastructures for
Cloud Computing and Big Data M**

Group Issues and Policies

Antonio Corradi
Academic Year 2025/2026

PARTITIONING AND GROUPS

One important aspect in acting in distributed systems is how to deal with **partitioning entities**, and of course, also **in groups**, of **replicated entities**

Generally speaking, all systems must insist on partitioning and replicating of entities

Partitioning – for Scalability

- Several entities must be in charge of the whole function to grant scalability of services and of support

Replication – for Availability

- Several entities must be in charge of the whole function to grant both availability of services and support

The two aspects are **strictly coupled**

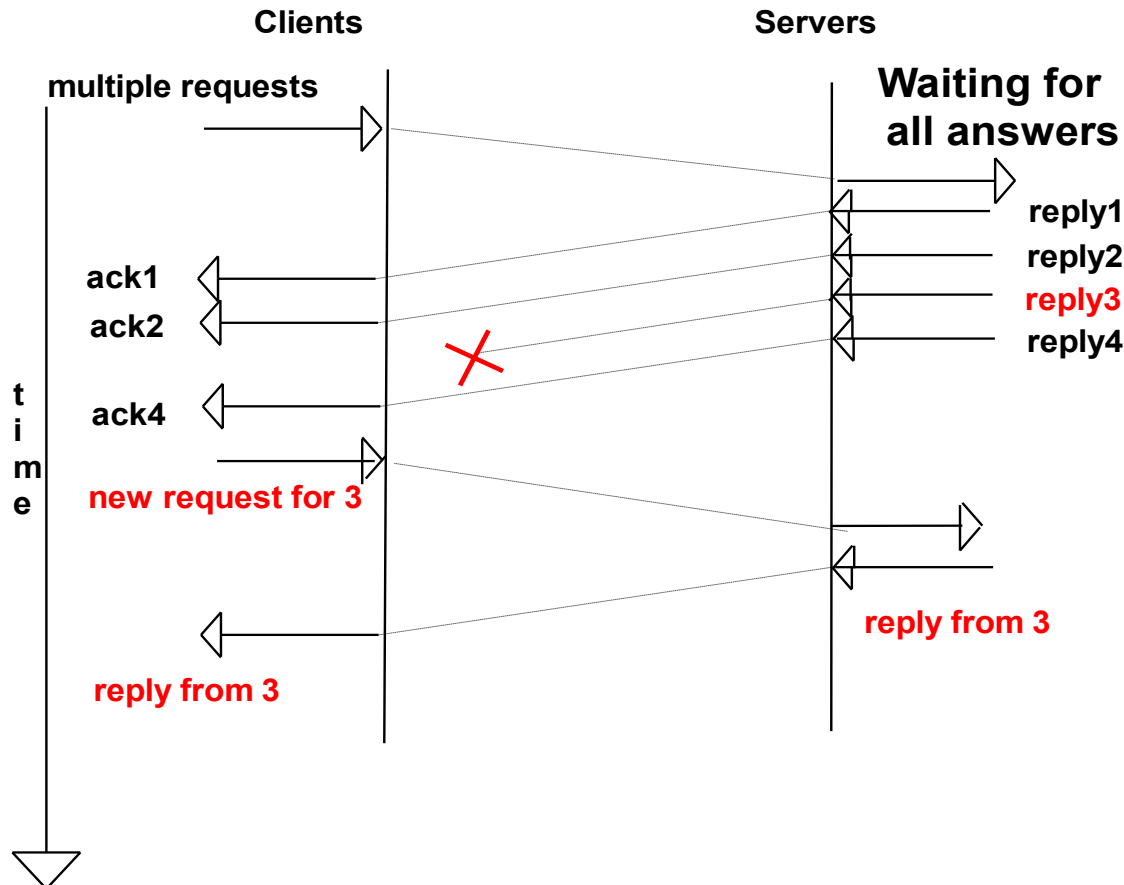
Staticity / **Dynamicity** of both aspects

GROUP COMMUNICATION

Semantics in Multiple communications: which protocols?

Use of selective retransmissions? How many times you resend?
When? How many answers?

Semantics deeply depends on these choices



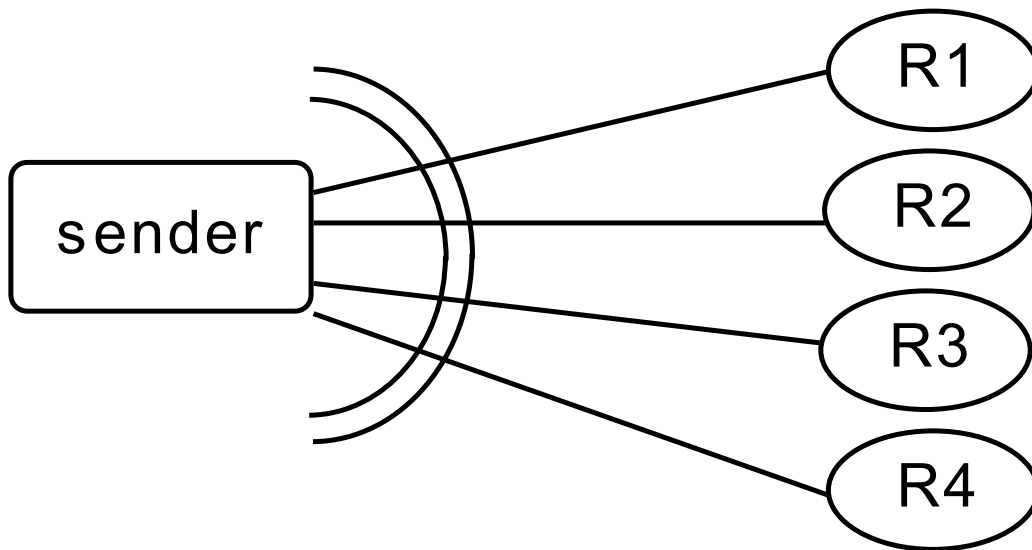
Global solicitation
VS.
Selective solicitation

Positive confirmation
VS.
Negative confirmation

GROUP COMMUNICATION

MULTICAST SEMANTICS

The **multicast** action could make the multiple group sending operations **atomic**, but they can try to associate a different and more suitable meaning



Even different senders

Motivations of the interest

- *fault tolerance and dependability*
- *object copy location within a system*
- *use of data replication and streaming*
- *multiple changes on group entities*

GROUP COMMUNICATION

TWO aspects of **MULTICAST SEMANTICS** are intertwined with each other and can be untangled (can be split)

Reliability

group members message reception

- reliable $\Rightarrow$ guaranteed delivery
- unreliable $\Rightarrow$ only 1 attempt (Chorus)

Atomicity

message reception for all group members

with **possibly different ordering for different actions**

The two aspects can and must be considered in separation

Essential Element

We must think not only to **the semantics of any single action**, but also to **message ordering in a multiple action occurrence** (and consider their synchronization)

RELIABLE MULTICAST

Reliability *can be achieved if* some occurrences cause no problems

- sender crash
- receiver crash
- message omission

Necessity of fault **identification** and **recovery** through *monitoring* of **multicast** and **group** actions:

- *check of every ongoing communication*
- *possible retransmissions*
- *removal of failed components*
- *protocol to re-enter in the group*

The additional costs for **identification** and **recovery** must be considered and *they apply in case of failures*

RELIABLE MULTICAST

Implementation decisions

- Dispatch all messages to the group members support and **delay** before passing them to the applications
timeout and **retransmission** (who checks the protocol?)
- **How long** to wait? problems with efficiency
- If **controller** fails?
Quis custodiet ipsos custodes? (Juvenal / Giovenale)

Hold-back → the support holds a message until it is sure that all previous others reached the destination in order

- In case of *dense numbering*, a message is delayed until all previous ones appeared (if is the message number 3, it must appear after the one numbered 2)

Negative ack → the support sends *a negative ack only in case of losses*, to identify those events (in a selective way)

ATOMICITY VS. No ORDERING

The other aspect of ATOMICITY is connected to semantic connected properties

with **atomicity** we focus on the **reception order of messages by any alive members of the group**

In **distributed systems**, sometimes, we are not so interested in obtaining a **very tight synchronization of copies**

No Ordering → the multicast messages coming **from any sending process** to **all receivers** can present a different ordering in any copy

The **No ordering policy** is very easy to support

It does not **have impact**, and you do not have to **synchronize copies in any way**, being them free of operating on their own

FIFO ORDERING

We have many situations in which we want to require a **form of coordination among copy scheduling**

FIFO Ordering → from **the same sending process** to **all receivers** for a sequence of successive multicast messages

In case of FIFO ordering, **two multicast messages from the same sender reach any group member in the same order**

- (m1 and m2 from S1) and (m3 and m4 from S2) reach everyone
- respecting sending order of the two senders many sequences are compatible (m1 m2 m3 m4), (m1 m3 m2 m4), (m1 m3 m4 m2), (m3 m4 m1 m2), (m3 m1 m2 m4), ...

We can use supports that can guarantee FIFO

Otherwise → **we need to achieve it!**

An easy way is **message numbering** for any **specific sender**

FIFO ORDERING LIMITATIONS

Compliance with *FIFO ordering* guarantees that every message to the group from the same sender (and its requests) **are received in the same order in which are sent from the group** (only related with same sender multicasts)

If we pass to consider more than one sender

- A sends a news N_a
- B receives the news and sends a response to N_b
- C receives first N_b then N_a (N_b before N_a)
- D receives first N_a then N_b (N_a before N_b)

We need to consider *cause/effect* relationships between different (two or more) senders

CAUSAL ORDERING

CAUSE-EFFECT ordering can connect events from different sender process

CAUSAL ordering → events that are correlated with a **cause-effect relationship outside the group** must be **acknowledged** by the group and the group must **achieve consistency** about them (**to be delivered to everyone**)

- **First the cause than the effect (Cause before Effect)**

In case of CAUSAL ordering, two multicast messages in the **causal relationship** must be considered in the right order from everyone (m1 and m2 from S1), (m3 and m4 from S2), (m1 causes m3)

So they must reach copies respecting FIFO and CAUSAL ordering

Many sequences are compatible (m1 m2 m3 m4), (m1 m3 m2 m4), (m1 m3 m4 m2), but **NOT (m3 m1 m4 m2)**

There are no supports that guarantee CAUSAL ordering

How can we guarantee it?

CASUAL ORDERING

Compliance with CAUSAL ordering

- guarantees that messages from different senders in **cause-effect relationship** are received in the causal order by the group
- for one sender only is similar to FIFO and it is easy to implement

Compliance with CAUSAL ordering do not catch **real world Internet (USENET) situations** that we implicitly take for granted in case of more than one operations

- A requests an action to Na; B requests an action to Nb
 - These actions are not related
 - C receives first Nb and then Na, D receives first Na then Nb
- So copies have different internal decisions of scheduling

ATOMIC ORDERING

No external relations imposes a scheduling, **but the group should act in a coordinated and reasonable way**, where all group members operate in the same order

ATOMIC ordering guarantees that **all messages are received in the same order by all group members** (so related actions can occur in the same order in all copies)

Often **no predetermined order** is likely so no need of previous agreement, **but it is necessary to dynamically agree on one and that order should be the same for all**

If a copy C decides to receive first Nb then Na, than all copies must follow that decision

- Nb may ask to compute an interest on a bank account,
- Na intends to make a withdrawal

Obviously, many different **atomic orderings exists** that we can consider with group operations

ATOMIC ORDERING

In a distributed environment the introduction and the enforcing of orderings is expensive (coordination between group entities or numbering support) and we tend to enforce it only when necessary

Minimum cost: no ordering → each group member works in a free and independent way

FIFO and CAUSAL ordering are constraints that we tend to enforce only for some specific events in the system

Partial orderings

ATOMIC ordering is an ordering that we tend to enforce on every event within the group in the system

Total or global ordering

ATOMIC ORDERING

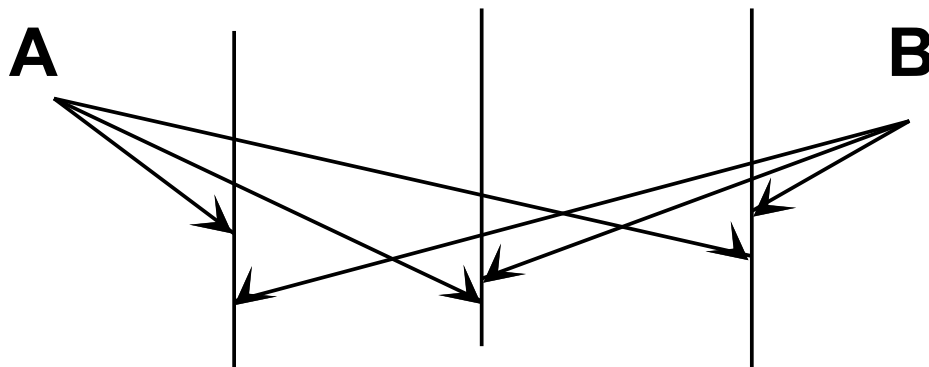
Obviously, given a group and a set of events coming from outside, we may have many different atomic orderings

How many?

ATOMIC orderings

Among many atomic orderings, some of them can follow **CAUSAL** and **FIFO** ordering, some only **FIFO**, some only **CAUSAL**, and some of other none of them

Costs for atomic orderings can be very different



MULTICAST ORDERING

FIFO multicast with ordering only from the same sender

- *messages from the same sender arrive with sender and ordering number and are delivered in that order (only for the same sender)*

CAUSAL multicast with causal ordering (logic)

- *messages arrive and are delivered to the group in order to respect the relationship if the event **A** causes the event **B***

Lamport ordering

ATOMIC multicast requires the same order (**any order**) for all messages to the group members

ATOMIC multicast imposes a total or global ordering for messages that arrive to the group and must be delivered with the same order to all correct members of the group

- **An Atomic order does not necessarily subsume the other two or anyone of them: any order can be decided inside the group**

MULTICAST ATOMIC ORDERING

ATOMIC multicast needs the same order for any message

A simple way to implement it is to have one **entry element (a front end)** inside the group and it is that one that orders messages while they arrive and imposes increasing numbers as order tags before sending them to all other members of the group

Every member receives messages and must send them in the agreed order to the application layers
(**linearizable consistency**)

Disadvantages of this implementation are

*Problems in case of **coordinator fault** (a SPoF)*

Unfair management: the coordinator neighbors or preferred ones can be favored and scheduled always before others (that have made their requests before)

The above is a low-cost solution, but very unfair

→ ***mobile coordinator (circulating token)***

Many other solution even more dynamic (Lamport, ring, etc.)

SYNCHRONIZATION

Synchronization means to impose **orderings on event** typically ...we pose some **constraints on temporal ordering of some events inside a distributed system**

It is necessary to provide a **consistent view** of the system to the **entire set** of communicating processes

Communication and Synchronization are often correlated, for example:

- *synchronizing sender / receiver* of a message
- check on cooperating activities
- *serialization of access to shared resources*
- N processes in access to a resource (mutually exclusive)

so, ***ordering on important events must be enforced***

CLOCK SYNCHRONIZATION

Synchronization by using **PHYSICAL TIME** and **PHYSICAL CLOCK** – typical on one local environment only

Unique time can be determined, if we assume that either

1. a **unique clock** is available on every node or
2. one **clock** for any node, and all of them perfectly in sync

*This work assumption is perfectly admissible in concentrated or limited systems, but **absolutely not feasible** and **difficult to be granted in distributed and global environments***

It has been defined **Universal Coordinated Time (UTC)** that is based on the transmission of the value and on local correction

Some systems are based on **coordination clock**

- A node verifies the time of all group members, computes the average and distributes it to all as the group time (**Berkeley time**)

CLOCK SYNCHRONIZATION

NTP - Network Time Protocol introduces a protocol based on UTC and on **synchronization to achieve an agreement on clocks**

NTP tries to overcome possible transmission delay of the common time through **statistical filtering policies** based on historic behavior of servers

- Starts with a higher **server hierarchy**, where every node transmit time to **lower-level neighbors** (its subtree)
- The **primary** nodes are more accurate and going farther from the **root**, accuracy decreases (of course)
- NTP tries to make actions to recover from **server** faults

The problem that can occur, **by using clocks not perfectly in synch, is that an event happened afterwards maybe labeled and considered before an event that precedes it in time** (that may produce a *wrong time synchronization*)

SYNCHRONIZATION IN LARGE SYSTEMS

Synchronization via PHYSICAL TIME clashes with the difficulties of guaranteeing synching of clocks and a high accuracy implies a high overhead and also is prone to errors

- **Precision requires to coordinate continuously the clocks, and it is impossible to avoid conflicts and clock drifting with limited overhead**

Typically, **distributed synchronization is not based on complex algorithms of physical clock agreement** but based on **different strategies that can restrict** the sync requirements, focusing only on a **subset** of global system **events**

- The idea is to **work on a subset of events** (considering only some **interesting events**) and to create an agreement only on them
- The assumption of a limiting focus and a reduced group can **limit the overhead and protocol cost**

SYNCHRONIZATION STRATEGIES

Several **Distributed Synchronization** Methods

Ordering of logical time of **Lamport**

We can use timestamps (time indicator) to label relevant events and to order them → logical clocks and "happened before" relationship

Token passing LeLann ring strategies

We can use authorizations, and the token can pass in a logical ring to order events

Events based on priority

We can use process priority to order correlated events.

Used in real-time systems and unfair (**special-purpose systems**)

LAMPORT RELATIONSHIP

In distributed systems, Lamport aims at **ordering some events (not all of them)**, by excluding physical time

Only **some events** are considered in the distributed system, with a scenario constituted by **processes** that have their internal history and can exhibit a behavior based on two kinds of events:

1. **local**: *local events*
2. **remote**: *interprocess events*, generated by sending messages from one process to another process (**send / receive events**)

We limit the actions of interest

The ordering must consider only some '*relevant*' events and aims at creating a simple **ordering policy**, on which to eventually establish a **correct** synchronization with **adequate costs** and **not very expensive to implement**

HAPPENED-BEFORE $\rightarrow$ RELATIONSHIP

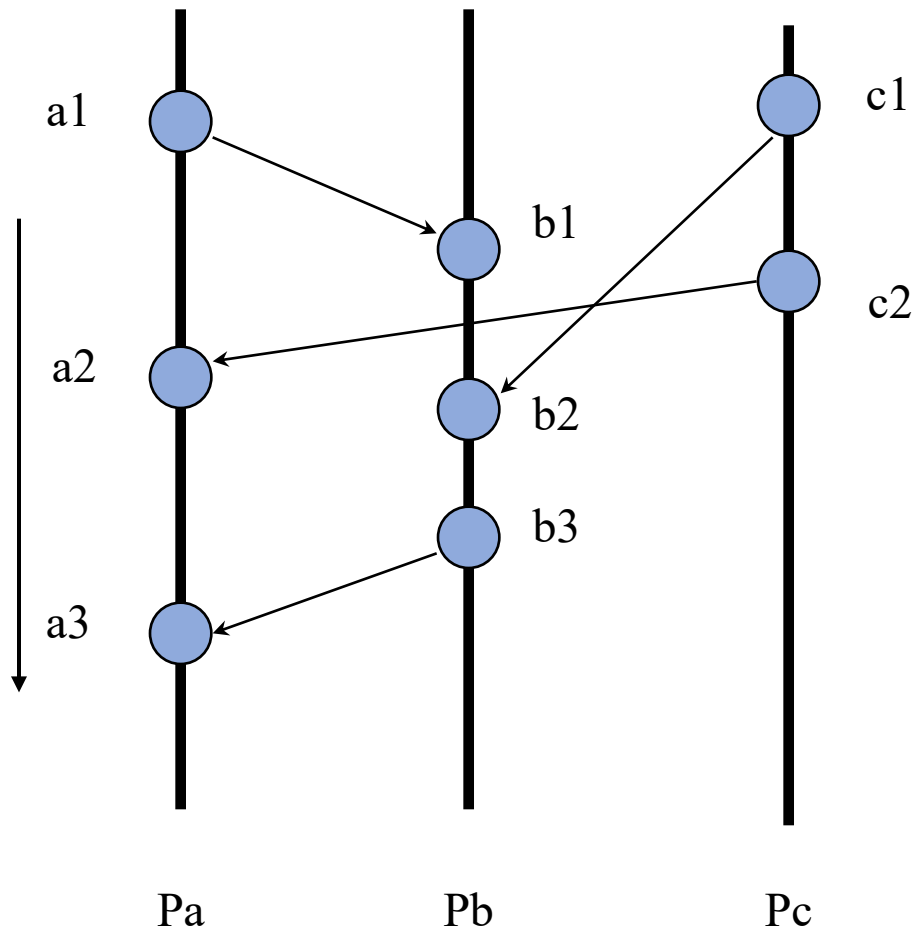
Events ordering for a **set of processes that communicate through message passing** based on cause-effect relationship introduced by process actions

1. If a and b are events of the same process and a occurs before b , then $a \rightarrow b$ (**local order**)
2. If a is the sending of a message of one process and b the receiving event within another process, then $a \rightarrow b$ (**communication interprocess order**)
3. If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$ (**transitivity**)

The relation $\rightarrow$ introduce a **partial ordering** in **systems events** and it exists **only among some systems events** and not assumed among all events (**it is not a total ordering**)

Two events are **concurrent** $\Uparrow$ if **not** $a \rightarrow b$ and **not** $b \rightarrow a$

HAPPENED-BEFORE RELATIONSHIP



$a1 \rightarrow a2, a1 \rightarrow a3$
 $a1 \rightarrow b1, a1 \rightarrow b2, a1 \rightarrow b3$

$c1 \rightarrow c2$
 $c1 \rightarrow b2, c1 \rightarrow b3, c1 \rightarrow a3$

Concurrent events
 $a1 \uparrow\uparrow c1, a1 \uparrow\uparrow c2, \dots$
 $a2 \uparrow\uparrow b2, a2 \uparrow\uparrow b3, \dots$

HAPPENED-BEFORE → RELATIONSHIP

The **happened-before** relationship allows to work in a distributed system in which only → is enough for ordering

- We do not assume a unique global clock (**global time**), but allow for a **set of local clocks (local time)**

We assume to work in an asynchronous environment, that makes possible any **transmission delay** for messages, *variable and unlimited, in principle, so higher* than any significant possible delay (but **messages are not lost**)

⇒ We may need several **ordering strategies**, also **global or total** to synchronize

*We organize a **logical time system** built on the → relationship that is **based on logical clocks** and not on physical clocks*

LOGICAL CLOCK & TIMESTAMP

We need to construct a **clock system (system timestamp)** to assign a simple indicator, a '**number**', to order events:

- *The **happened_before** relationship is only **partial***

We define a function **TS(i)**, a *logical time-based function (timestamp)* that must assign a value to any relevant event

If $a \rightarrow b$ in the system, then the logical timestamp of events must respect the law $TS(a) < TS(b)$

If need a **clock condition**, we want to infer the global logical clock function **LC** for system events related to processes P_i

Clock condition (Logical Clock - LC)

Given a and b , if $a \rightarrow b$, then $LC(a) < LC(b)$

NOTE: it is not true that, if $LC(a) < LC(b)$, then $a \rightarrow b$

LOGICAL CLOCK & TIMESTAMP

Any process P_i has a logical clock $LC_i(c)$ (an integer counter)

C1. For $\forall a$ and b , if $a \rightarrow b$ inside the same process P_i , then $LC_i(a) < LC_i(b)$

C2. For $\forall a$ and b , if a is the sending of a message in the process P_i and b the reception in the process P_j , then $LC_i(a) < LC_j(b)$

I1. Every process P_i increments LC_i between any two events

I2. For $\forall a$, sending of a message in process P_i , the message contains a clock as timestamp $TS = LC_i(a)$

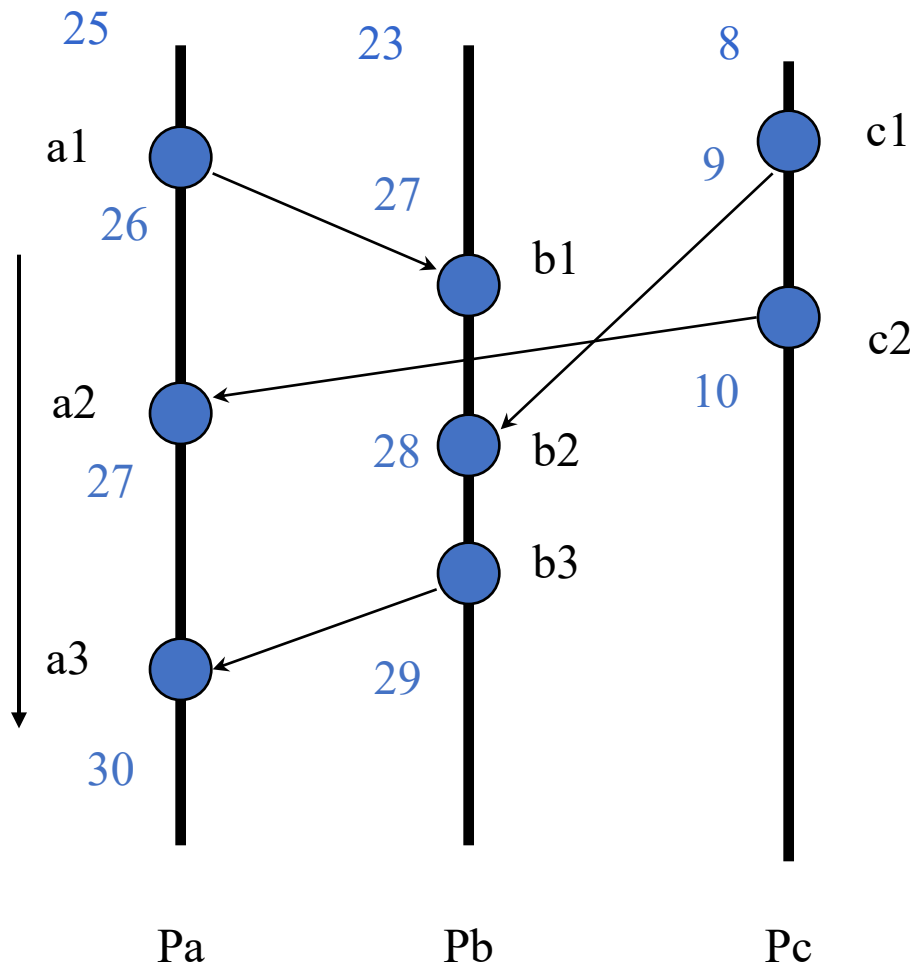
I3. For $\forall b$, reception of a message in process P_j , the process put the logical clock at the greater value between current clock and timestamp $LC_j = \max(TS_{\text{received}}, LC_{i\text{current}}) + 1$

2 clock conditions and 3 implementation practices

These rules introduce a partial order relationship

There are many concurrent events $a \uparrow\uparrow b$ with equal timestamp

WHO DOESN'T RECEIVE, DOESN'T UPDATE

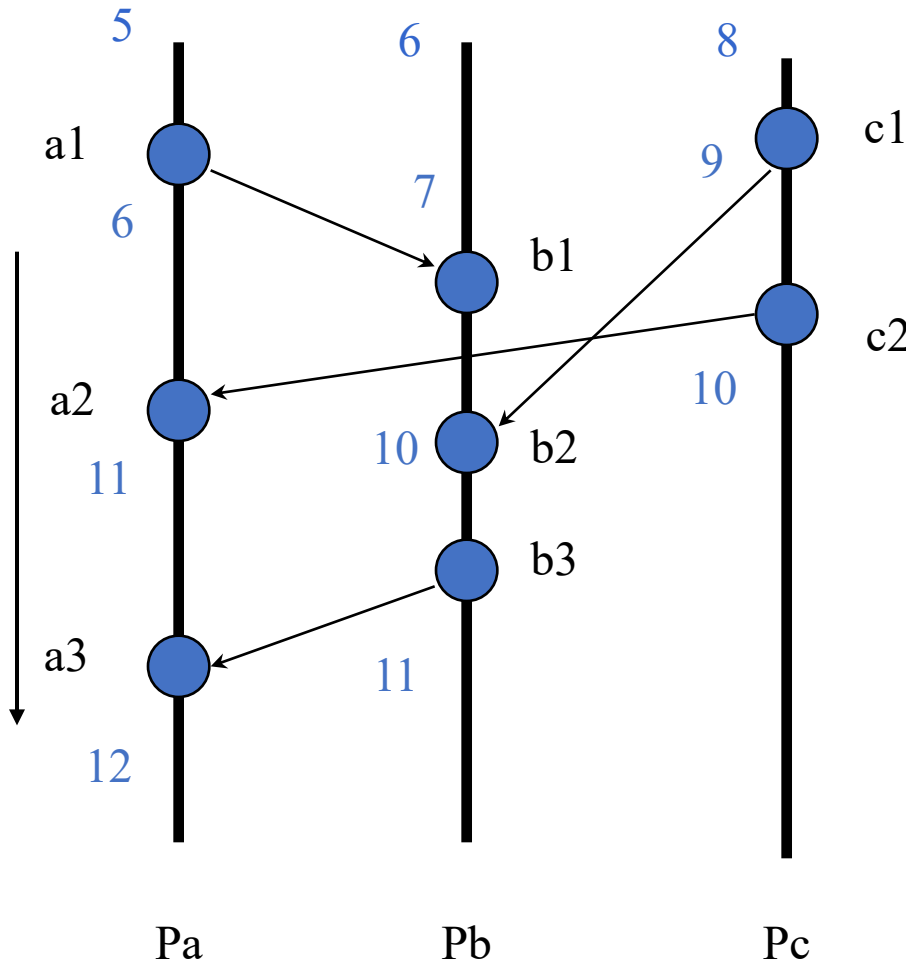


The → relationship allows to order events according with a logical cause-effect relationship

but the sender **has initiative and forces the update the logical clock of the receiver**, but not **its own...**

(it is the receiver that has to update clock to sender, with a transmission eventually)

HAPPENED-BEFORE → PARTIAL



The → relationship **allows to catch cause-effect ordering of events**

But ... it also make you assume an ordering of events even not in the → relationship

- Concurrent events in real world – such as c1 and b1- are considered one after the other ... so in sequence

What is the relationship between the **b2** and **c2** events (**same timestamp**)?

ORDERING AND REALITY

The Lamport relationship → is a **logical** one and it is **loosely connected with the real world**; it cannot be considered a physical world relationship (it does not respect 'reasonable' human behavior)

In general, **who receive messages updates its time**

Those **who do not receive messages** may maintain a **very low timestamps** and are **not forced to sync logical clocks** (so their timestamps can be very favorable)

Causality problem in clocks: **Two events considered by Lamport in a causal relationship may not be related at all**

Hidden channel problem: **If a process can use an external and non mapped channel to communicate (hidden channel), that can lead to a situation that do not respect cause / effect relationship**

The **effect** in **real world** can have a timestamp lower than the one of the **cause**

TOTAL ORDERING AND $\Rightarrow$

Sometimes it is necessary to introduce some **conventional total order relationship** among **all process events** in the system

These cases are dealt with by a **global order relationship** $\Rightarrow$ *between all system events* that is based on logical clock and on the partial ordering of $\rightarrow$

total order relationship $\Rightarrow$

If a is an event in process P_i and b an event in process P_j , then

$a \Rightarrow b$ if (and only if):

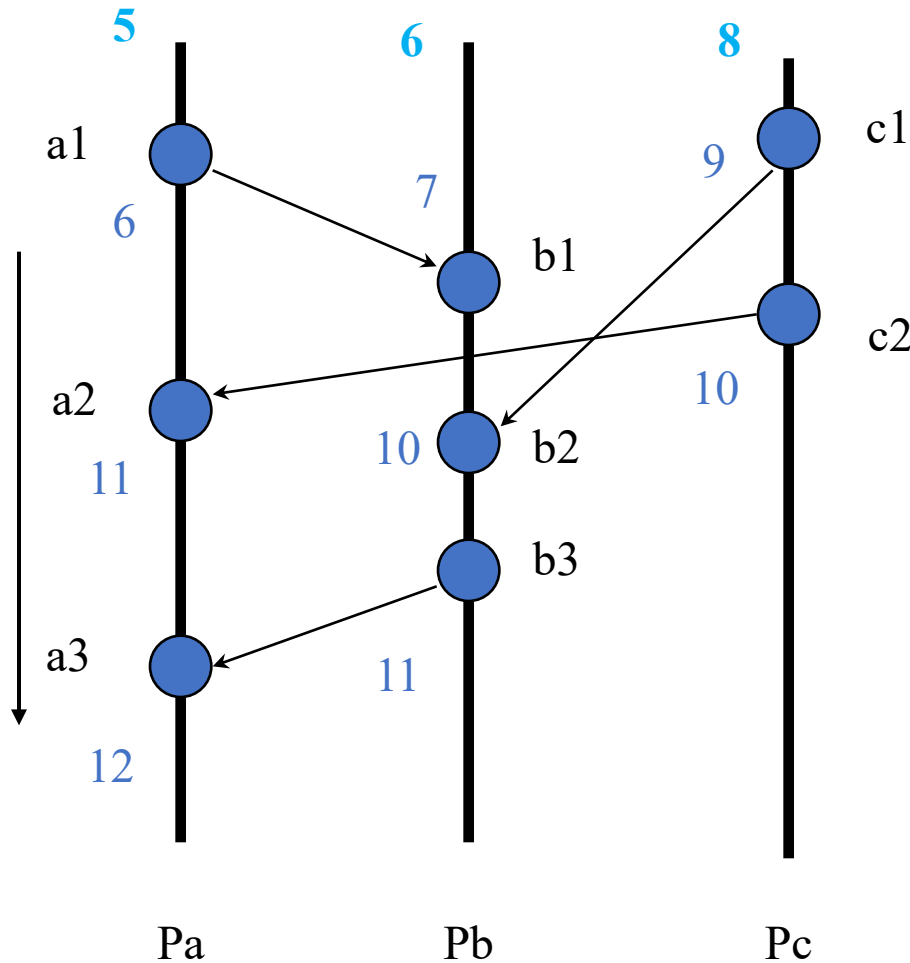
R1) $LC_i(a) < LC_j(b)$ or

R2) $LC_i(a) = LC_j(b)$ and $P_i < P_j$

The total ordering assumes that in case of events of the same logical clock, **there is an order between all process events**

It is possible to use $\Rightarrow$ to define a univocal and simple ordering to create synchronization upon

HAPPENED-BEFORE $\Rightarrow$ TOTAL



The $\Rightarrow$ relationship orders any pair of events

It makes possible to consider two events one after the other while they are instead concurrent in real world

$c2$ and $b2$ are managed as in sequence, by considering first process **P_b** , then **P_c**

→ AND REALITY

Let us keep in mind that
Happened before → is only one way
and not bidirectional

Given a and b , if $a \rightarrow b$, then $LC(a) < LC(b)$
it is not true that, if $LC(a) < LC(b)$, then $a \rightarrow b$
So you cannot infer that
 $LC(a) < LC(b)$, then $a \rightarrow b$

Sometimes we need a closer relationship between
Lamport model and reality extending the clock models
Toward a two-way relationship

Bidirectional ordering via some implementation

VECTOR CLOCK ORDERING

There are other strategies

it is possible also to consider **vector logical clocks** or **Vector Clocks** to order events in a process set

Processes must maintain a vector of all known clocks of processes and use that in communication

Every process keeps its **timestamp** and a **vector $V_i[k]$** of integers of a **dimension of the number of processes**

A **vector clock** element **$V_i[k]$** contains information on **what a process knows about the clocks** of other processes

The process P_i in the vector keeps:

1. $V_i[i]$ its timestamp (index i)
2. $V_i[k]$ the timestamp of any other process P_k at its knowledge

So the **data structure** is more complex for processes and also for the protocol to communicate and update the vector

VECTOR CLOCK PROTOCOL

The **Vector clock** update protocol is based on the steps:

1. Every process P_i increments $V_i[i]$ between two events
2. For $\forall$ sending of a message to process P_i , the message contains the whole vector clock at best knowledge of P_i after incrementing its own $V_i[i] = V_i[i] + 1$
3. For $\forall$ reception of a message, the process P_j increments its own $V_j[j] = V_j[j] + 1$ and updates its vector according to

$$V_j[k] = \max (V_j[k], V_i[k])$$

The receiver obtains information on the logical time of the sender process and also on time that it knows of all others

Vectors clocks allow a better information propagation and permits a wider information exchange and diffusion
(sometimes matrices are used)

VECTOR CLOCK PROTOCOL

The **logical clocks of the receivers are updated**, when a message is received

The main cons is that events not in the $\rightarrow$ relationship can be taken as if they were

The **vector clock** protocol instead **pays the cost** of the propagation of the entire vector and **requires adjustment** of the entire vector at the receiver

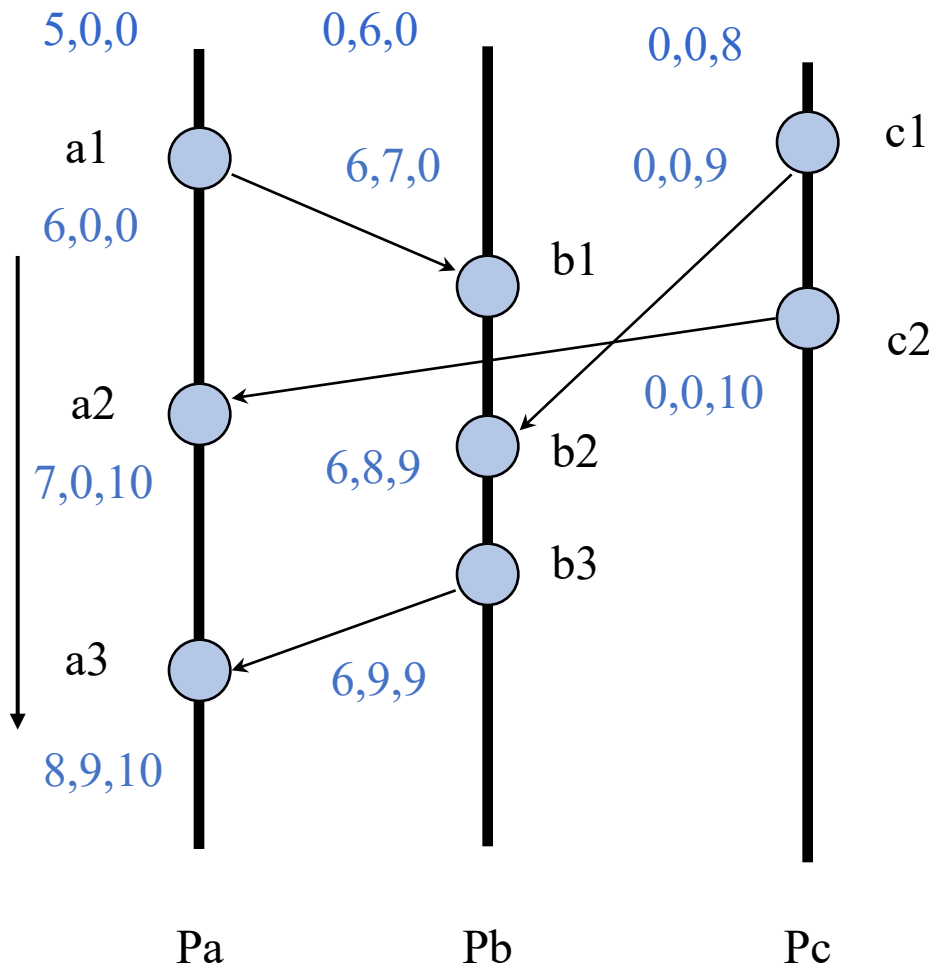
The **vector clock $\rightarrow$** requires to apply to all dimensions of the clock, ... so it becomes more significant and the relationship is bi-univocal

With vector clock algorithms

The events in $\rightarrow$ are recognized to be in cause effect relationship and

the events not in that relationship, i.e., concurrent events $\Uparrow$, are recognized **not to be in the $\rightarrow$ cause-effect sequence**

VECTOR CLOCK



With **vector clocks** we can identify if two events are in a **real cause-effect relationship**

Not only **events in relationship are tagged and ordered**, but other **events that are not causes effects are recognized as such**

Concurrent events in real world - $c1$ and $b1$, $a1$ and $c2, \dots$ - are **not considered** in the **cause effect relationship**

SYNCHRONIZATION

The **simplest synchronization** case is the scenario of a set of processes that have to access a **resource** in a **mutually exclusive way**

We assume that every process must access to the resource for a limited time and must release it after usage

OBJECTIVES:

Safety: only one process at a time can have access to the resource

Liveness: every process that has done a request receives the access after a limited delay

Fairness: different requests must be managed by a fair policy

Obviously, we have many ways to realize it

We **exclude fixed priorities** that are **unfair** and can **cause starvation**

SYNCHRONIZATION

For synchronization, we can follow an approach based on a single central **coordinator process**:

- an **approach completely centralized** considers a unique **coordinator process** known to all other processes (all participants must not know each others – C/S model, but they know the coordinator)
- every process that intends to access the resource sends the request to the **coordinator** and after usage, **notifies it**
- the coordinator process decides the scheduling of the resource accesses by using its policy to grant mutual exclusion

Obviously, the coordinator can decide different policies (FIFO management or others)

We assume that the **coordinator receives all requests sent and queued in a reliable way (but with any delay)**

RESOURCE COORDINATOR

Protocol of the coordinator:

1. a process when it intends to access to the resource **sends a request message** (*request*) to the coordinator
2. the coordinator serves its request queue and it is free of deciding the request to reply to (*reply*). Obviously, it must send **only one reply** to one request at a time (typically FIFO)
3. when receiving the *reply*, the process can use the resource and at the end, must send a **release message** to the coordinator (*release*), that can decide to reply to another request, etc., etc.

3 messages for every access to the critical section.

There are several disadvantages stemming from the centralized and unique role of the coordinator

The case of coordinator fault and of its potential unfairness

Differentiated delays in reaching the coordinator

LAMPORT SYNCHRONIZATION

Lamport proposes a decentralized solution without single failure points

A set of **N processes** that must access to a **single resource** in **mutual exclusion**, without assuming any **centralized** role and trying to grant that requests are served in order (in a **fair** way)

Participant processes must **only examine their request queue**

Processes **exchange messages** between each others to obtain synchronization and must use **Lamport clock relationship** (up to $\Rightarrow$ relationship)

Assumptions:

- *the connection between processes is **complete and direct***
- *messages between processes **must arrive in FIFO** order*
- *messages can be delayed but **not lost***

LAMPORT PROTOCOL

Use of logical clocks and Lamport relationship

Every process has a **local queue** of received messages, in which **messages are queued in order of timestamps ($\Rightarrow$)**

For every process, the local queue initially contains the message $T_0:P_0$, lesser than every clock in the system

- clock is considered a logical time, specified by the couple of **an integer and the process identity** that owns it
- every message has a timestamp that depends on both components (**process and logical clock**) to allow fair total ordering

A process that decides to access to a resource must execute a **global coordination protocol**

Every process must know any other one and faults are not expected (N processes in order of index compose a static group)

LAMPORT PROTOCOL

Protocol for the group

1. The process P_i sends the **request message** $T_m:P_i$ to every process (even in its own queue) to signal its intention to access to the resource
2. At message $T_m:P_i$ reception (already in its queue), the process P_j sends a reply with its updated timestamp (Lamport $\Rightarrow$)
3. *The process P_i can use the resource **if in its local queue**:*
 - It has the **request $T_m:P_i$ ordered before any other request** of other processes ($\Rightarrow$ [relationship](#))
 - It has at least **one** message coming from **any other process (N-1)** with a **timestamp successive** to $T_m:P_i$

At the release, P_i removes the messages from its queue and sends a **release message** with its timestamp to every process
4. Every process P_j receives the **release request** and removes the request message from its queue

RESOURCE SYNCHRONIZATION

That solution grants that every process that executes the protocol can receive the resource with a limited time delay, if every process respects the constraints

Let us note that the process that has requested to access, enforces and waits a coordination with any other participant

- ***Every request sent message requires a response from all others***

While waiting for messages from one process, requests may come from other processes that may precede the concurrent one. Once they arrive, they are queued and sorted by timestamp.

Every process queue is ordered, and so a process can pass only when 'previous' requests have been served already.

At least (N-1) messages sent and the same number received before entering (N-1)

(N-1) to exit

SYNCHRONIZATION COST

The synchronization worst case is when all processes want to access the resource at the 'same' time

- *In case two processes make a request, they separately agree on the fact that first to enter is the one with the lower timestamp so there cannot be conflicts*

The algorithm occurs **without centralization**, but in a **completely distributed way**

For every **action** on the critical section the **number of exchanged messages** is (considering a **possible broadcast** as N-1 messages, unless you can obtain lower cost)

Number of messages $3 * (N-1)$ **or N-1** and **2 broadcasts**

We have a **high cost** due to decentralization

Heavy assumptions on the static group and no faults

OTHER M.E. PROTOCOL

Ricart & Agrawala protocol (R.A.)

1. Process P_i sends the **request** message $T_m:P_i$ to any process (even it its queue) to signal its intention to access to the resource
 2. At message $T_m:P_i$ reception the process P_j :
 - **sends an immediate approval reply** if it does not **need the resource** or if the **requester has a higher priority**
 - **delays its approval reply** if it is **using the resource** or it has **already asked to enter and it has a higher priority**
 - 3) Process P_i access the resource only if receives **N-1 approval messages**
 - 4) At release, process P_i must send approval to all arrived requests
 - 5) *The requests (and replies) are deleted after approval*
- Only one process can have **N-1 approval** responses and only one process can access the resource at a time

R.A. - SYNCHRONIZATION

For every action in the critical section, the number of exchanged messages is (a possible broadcast costs as N-1 messages)

Number of messages $2 * (N-1)$

So, there are N-1 messages from requester and N-1 from everyone else. **Difficult to foresee a coordination at lower cost**

These algorithms are based on **variations of Lamport relationship**

- are **completely distributed** (no unique manager)
- *fair and free from deadlock and starvation*
- *they may have high costs* in terms of exchanged messages for coordination
- *they may have high costs* due to decentralization

Heavy assumptions of messages **not lost** and **static group** without faults

Are those so heavy??

ATOMIC MULTICAST

Distributed implementation of **atomic multicast** can be less centralized than the obvious one with a unique coordinator

CATOCs (CAusal & Totally Ordered Communication operations Support) based on a by-need dynamic *coordination of a set of managers* that decide internally the request order

The group **does not have a unique central manager**, but coordinates on need and create a unique vision: it is possible to **have a manager selected for every request** that negotiates with others and obtains all the requests to synch with others

Realization not so **scalable** and **implementations** of different **efficiency (?)** or **at least efficient only in specific cases**

- *Availability of a **broadcast at a low level** can solve many implementation problems and **enhance efficiency** (we also need a support that grant the assumption of not losing messages, connecting all processes, ... etc.)*

ATOMIC MULTICAST - ISIS

ISIS appeared in the 90s ... for CATOCS in UNIX

ISIS is system based on **groups** with **active replication** and with necessity of a vision with **different degrees** of **coordination** of group components

The system obtains coordination **with many different forms of group multicast (called broadcast)** for the same group

Many different **multicast forms** are available (**BCast**):

FBCast (**FIFO BCast**)

CBCast (**Causal BCast**)

ABCast (**Atomic BCast**)

GBCast (**Group BCast**)

Providing also support to the case of no copy coordination.

Any **operation need a manager**, typically **dynamically chosen** according to any kind of policy (vicinity, rotation, ...)

ATOMIC MULTICAST - ISIS

ISIS ABCast (Atomic BCast) and its cost $3*(N-1)$

CATOCs uses a queue for every corresponding component of the group and Lamport relationship

The messages are tagged with an *initial arriving timestamp* and are only considered (and processed) if **labeled as *final*** in the **right order for Lamport** relationship

Every arrived message requests a coordination phase of the manager (and *hold-back*) to determine the final timestamp to be used by all copies to execute in the correct order

A group should be capable of **operating with all the ordering policies for any request**

ATOMIC MULTICAST - ISIS

The coordinator receives the message:

- **labels it** and **sends it to all others** (with *its timestamp*). anyone else labels the answer with its **timestamp** based on its time (clock) and sends the answer back with its timestamp
- **labels it as final** with the received **highest timestamp** (is that choice and policy *necessary*?)
- **resends the message with the final timestamp to all others to communicate the final decision**

Any in the group has all finalized messages in the same order in its queue so it can drive in the same order the execution

Problems: delay and overhead - cost in messages of $3 * (N-1)$

ATOMIC MULTICAST

ISIS ABCast (Atomic BCast) achieves the **total ordering of messages** for a group toward a **coherent group vision**

- The group must reach an **internal agreement** that can also be **not compliant** with the **external timestamping** (not respected)

Group members cannot operate on one request until it is sure that the message:

- **has been seen also by everyone else** (arrived to anyone)
- **has been ordered with respect to any other message for the group** (arrival order)

The group is achieving consistency in operation ordering and, so, atomicity and global order is guaranteed:

- *other messages not yet arrived, maybe sent before these, will be considered only afterwards by the entire group*

And if we must guarantee **causal multicast**?

How do we do that? It is more or less complex?

CAUSAL MULTICAST - ISIS

ISIS CBCast (Causal BCast) is a **partial ordering**

- CBCast tends to consider only **some external events** that are to **be ordered with one another**; all other events can be ordered differently by group components (so limiting costs and coordination)

ABCast tends to impose an order based on timestamping decided **inside the receiver group** (**internal** event ordering strategy)

BCast requests a behavior **decided outside the receiver group** that must detect **cause-effect** relationships by inferring it from **timestamps arriving from outside** (**external** event ordering)

The Causal Broadcast assumes a **coordination** between senders that must update their “**logical clock**” and send information to receivers (requests queued by **sender timestamps**)

Group members must respect that external ordering

*If a **cause** would not reach the group before processing the **effect**? Either undo or error are necessary (!!)*

OTHER MULTICAST - GBCAST

ISIS GBCast (Group BCast)

The **group of processes can dynamically change in cardinality**, so it is possible **to join or to leave the group** for different reasons (possible **group inconsistencies and problems**)

*For every **concurrent multicast**, the message arrives in two states:*

- *to every member **before** group changing*
- *to every member **after** a group changing*

for a consistent ordering of any BCast, **either before or after**

we need to define a **new operation** for tracking the dynamic behavior of the group

GBCast makes possible to **order all BCasts**:

any **GBCast message must be either received after every previous BCasts in the processes** (or before - in a consistent way)

OTHER MULTICAST - ISIS

ISIS GBCast (Group BCast)

The **GBCast** was introduced to design a correct **dynamic grouping**, with no need to stop and reconfigure and work no stop with no problems

GBCast requires an **automatic monitoring support** for **group variation events** (any insertion and extraction trigger one GBCast)

When **anyone detects a failure** of a copy (or a new copy to be inserted), the **GBCast is issued to all copies** to make them aware of the reconfiguration

The ***group support*** is in charge of of invoking it

Every **group member uses a table** for **other members**: that table is updated by any **GBCast** (so all other BCasts can be aware of it and consistently ordered)

JGROUPS – RELIABLE MULTICAST

JGROUPS

Java Support for reliable multicast and for group concept
(Designed in Java and with user defined proprieties)

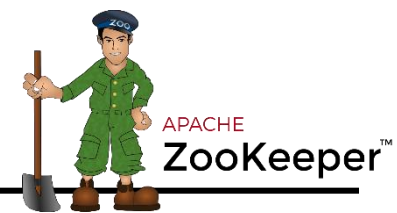
JGROUPS starts with a **transport level**, either not connected or connected, and it is also possible to work with JMS (Java Message Service) for message specifications

The goal of JGROUPS is **group and message delivery ordering**: it proposes a **reliable** implementation, intended as delivery with **message retransmission**, with most common different ordering: **Atomic, FIFO, Causal**, etc.

For the **group property**, groups are **dynamic** and managed in membership: **every group element benefits from group messages**, both from outside that from inside the group

*Possibility of **security**, like encryption and other secure support protocols*

APACHE ZOOKEEPER



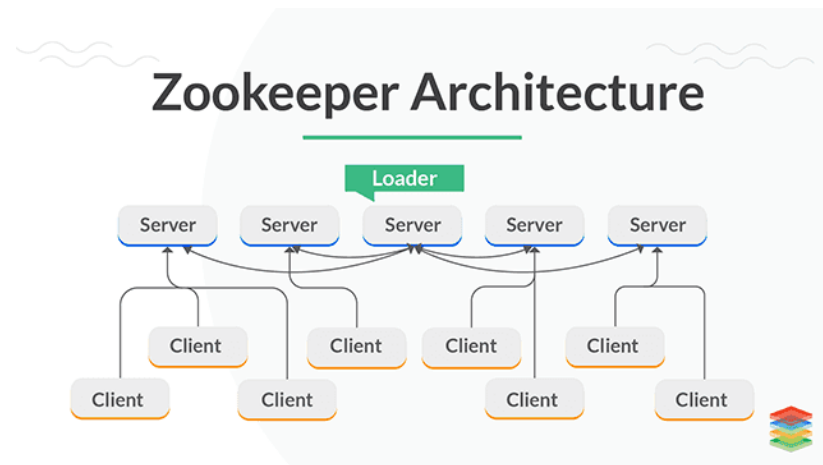
There is more and more need of containing data in a distributed way and with good properties as a commodity

Reliability, availability, efficiency, ordering, ... QoS

Zookeeper uses **replication over several z-nodes**

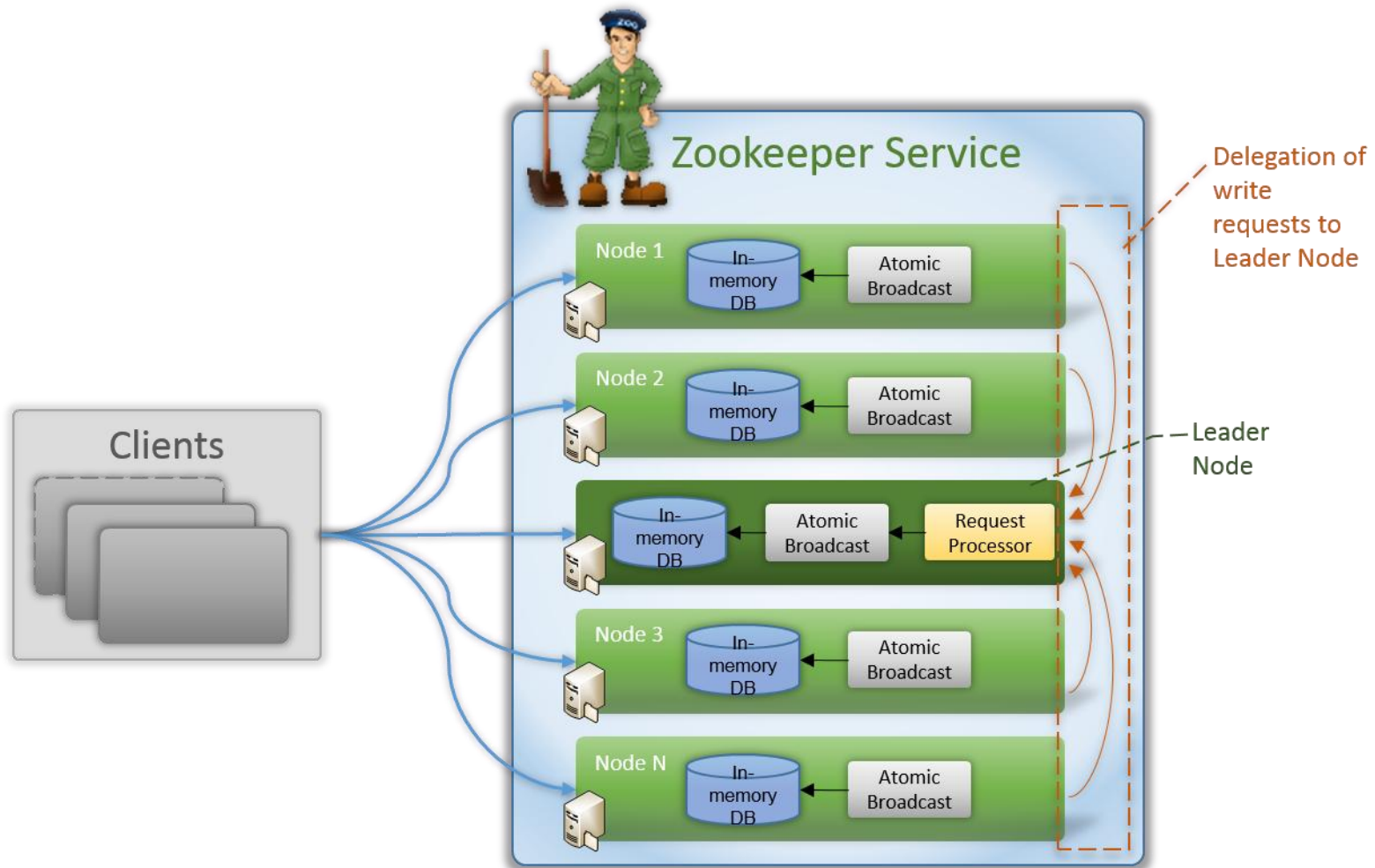
ZooKeeper is a **centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services**

Distributed applications can use all of these services in some forms or another



APACHE ZOOKEEPER

The internal architecture is very modern and fault tolerant



ZOOKEEPER

ZooKeeper is an open-source **Distributed Coordination Service for Distributed Applications:**

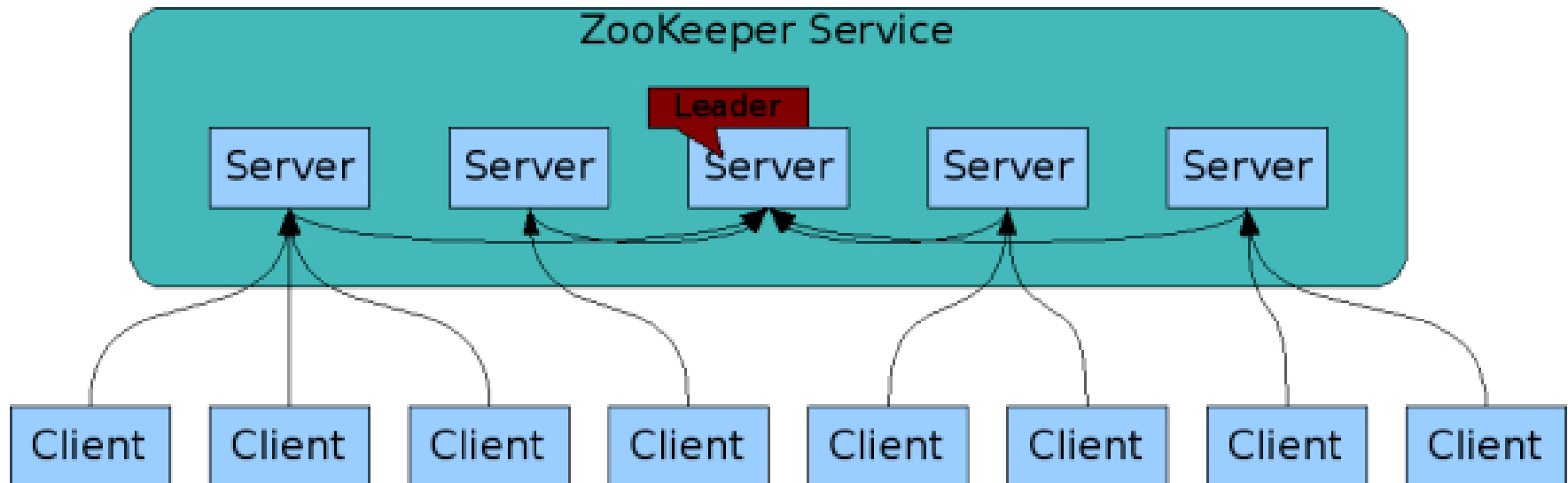
- can propose a unique **memory space with very fast access in **reading** and **writing** with some quality** (QoS: replication is paramount and dynamicity too)
- exposes a **simple set of primitives** to implement **higher level services for synchronization, configuration maintenance, and groups and naming**
- relieves distributed applications from **implementing coordination services from scratch**

The **data model** is shaped after the familiar **directory tree structure of file systems**, and it runs in Java with bindings for both Java and C

ZOOKEEPER

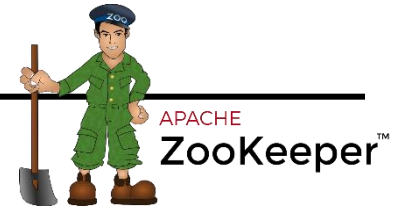
ZooKeeper is used as a unique access space with very fast operations to read and write data with many supported different ordering semantics (FIFO, Atomic, Causal, ...please note those orderings 😊)

Data are dynamically mapped over several nodes and their location can be dynamically changed and adjusted without any actions of clients

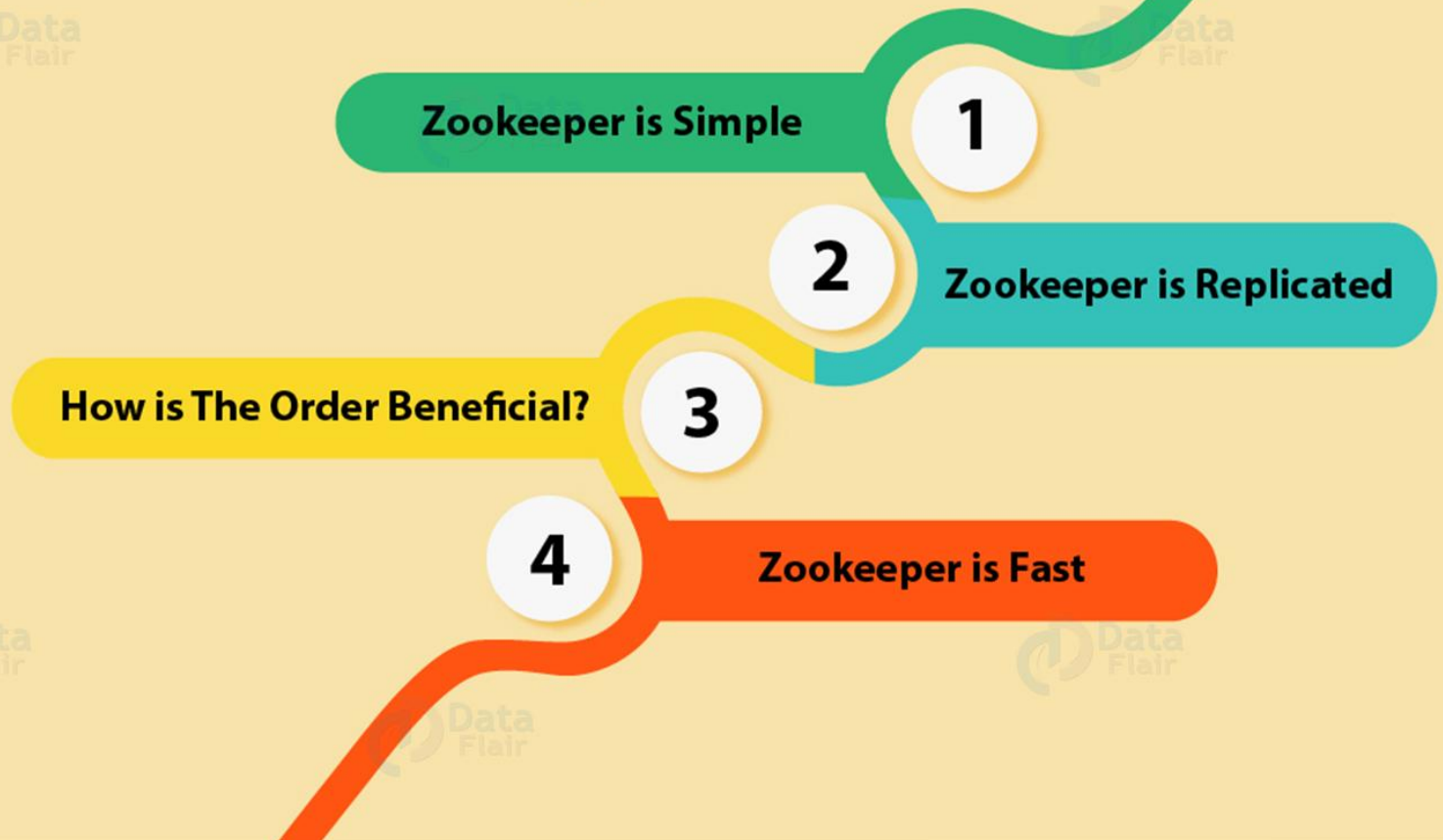


APACHE ZOOKEEPER

ZooKeeper flagship requirements



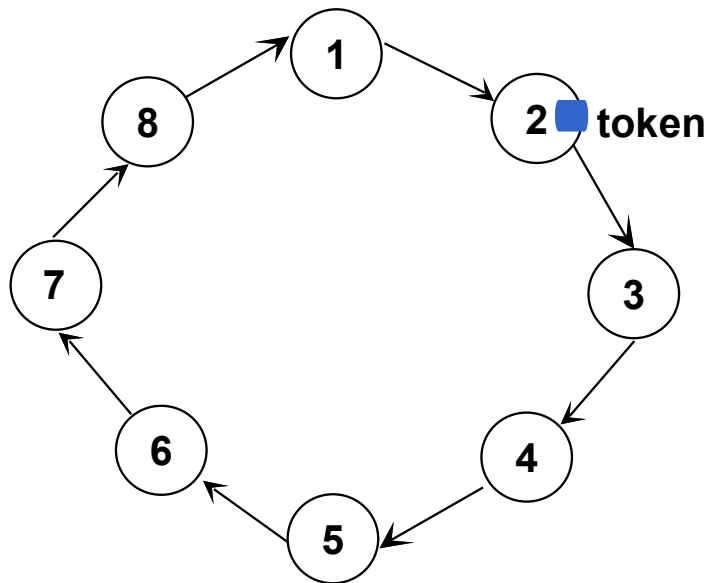
Design Goal in Zookeeper



SYNCHRONIZATION BY USING TOKEN

To overcome the problem of one **central coordinator**, the synchronization can be deployed by changing the role of the coordinator and varying the responsibility

The synchronization is **associated** with a **token**, dynamically passed between N different participants



The nodes are organized in a **logical ring (ON)**, where every node knows the next one (successor and predecessor)

Every node acts as the group manager when it owns the **token** that must keep for a while, then must pass to the next one

The token **circulates among** the N different participants (**time of comparison of token**)

SYNCHRONIZATION IN A RING

A **logical RING** connects all N participants and the token current owner is the manager of Mutual Exclusion

Protocol to access to the resource: who has the token,

- verifies that it is the expected recipient and
- uses the token for a time period with a maximum detention (it **manages ME to access resources** for all N nodes)
- after detention, forwards it to the following node

If the token moves in the ring in **one direction** only

- **Only one process at a time can access ME resources**
- **No conflicts can arise**
- **Starvation is not possible**

Number of messages N for a complete token turn in the ring

The working scheme is typically proactive: the token must circulate even no requests occur

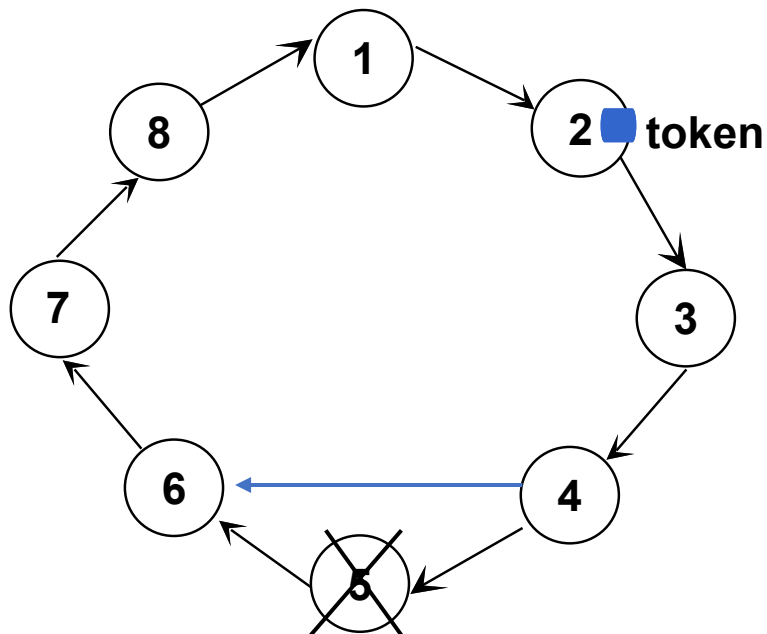
Problem if the token is lost (*failure of the node that has it*)

RECOVERY IN A RING

The **RING** architecture allows to execute **very simple recovery** algorithms in case of **single fault with no token problem**

Obviously, any node must execute some local neighbor correctness checks to **re-create the ring and overcome failure in case of neighbor failure** (node 5 shortcut)

Any node must also know the further following / preceding node



In this case, **node 4 and 6** checks its successor or predecessor and re-establish the local situation of the ring

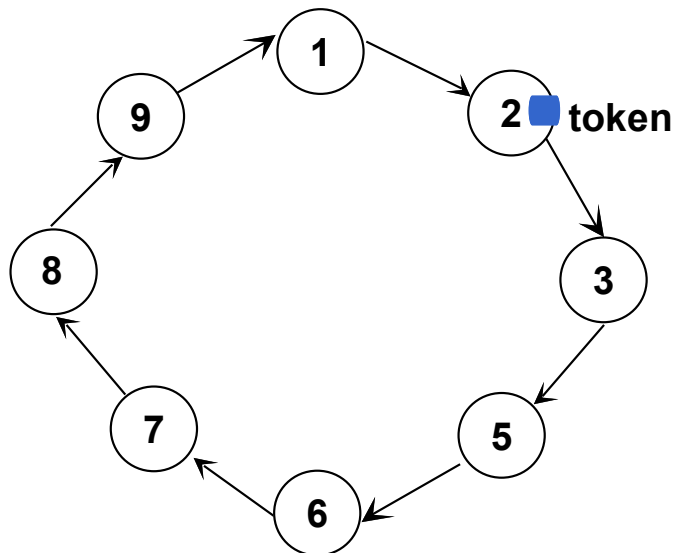
No problem to the token is caused

TOKEN RECOVERY IN A RING

The case of losing the token or having more than one must be avoided (since they are unsafe for ME).

- In case of **failure of the node that holds** the token, it is necessary to **regenerate it**
- **Token loss must be prevented** (due to fault on manager node)

Every node, taking part in the ring, activates a timeout interval that is reset at token return



In case the **timeout** is triggered, the node starts a **recovery** procedure to regenerate the token

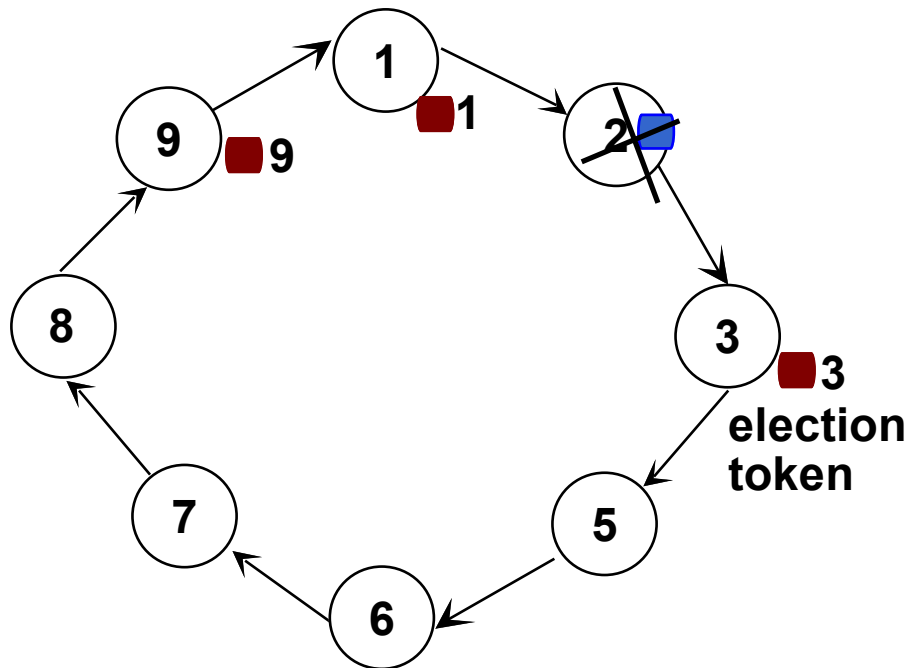
Note that **more than one node** can start the recovery procedure

TOKEN REGENERATION ELECTION

The **RING** architecture shows **very simple recovery strategies** in case of potential **token loss**

Obviously, any node is in charge of monitoring and attempting to start a recovery token protocol

Several timeouts may trigger the attempts of several nodes:
use of **node priority** for the decision (**Election protocol**)



In this case, the token can be regenerated only by the node with highest priority among the considered ones (here **the number 1**)

The election token becomes the new token

ELECTION IN A RING

Election protocol to decide who must become the manager (with a unique new token) based on **static priority of processes**

- At timeout, the process creates an **election token (ET)** with his name and enter an **election state** until the token returns
- If the process receives the normal token before the generated *ET* is back, the election is considered useless and terminated (***ET destroyed at return***)
- If the process receive an *ET* from another process, it is registered on an *election list* together with *identity of process* that generated it, and the ET is passed forward in the ring
- if it has already generated an *ET* token, verifies the *static priority* and decides who has ***highest priority*** in the election
- If the process receive its *ET*, removes it and verifies the registration list. The process generate a new token, only if the it is the node with ***minimum index (top priority)*** inside the registration list

ELECTION PROTOCOLS

The **election protocols** are used any time an agreement among **participants** must be found **without a predefined policy**

They are typically necessary in case of **fault** and **recovery** in a **group** to obtain distributed and easy agreement on a decision

In many cases, it is based on a potential **static order of participants** (**COST of the ELECTION PROTOCOLS?**)

BULLY algorithm:

every participant P_i that detects necessity of an election (event local to everyone) or a recovery toward a management role can do it

Three types of messages are considered

- message **Election**
- answer **Answer**
- announcement **IAmCoordinator**

How many phases there are in election protocols?

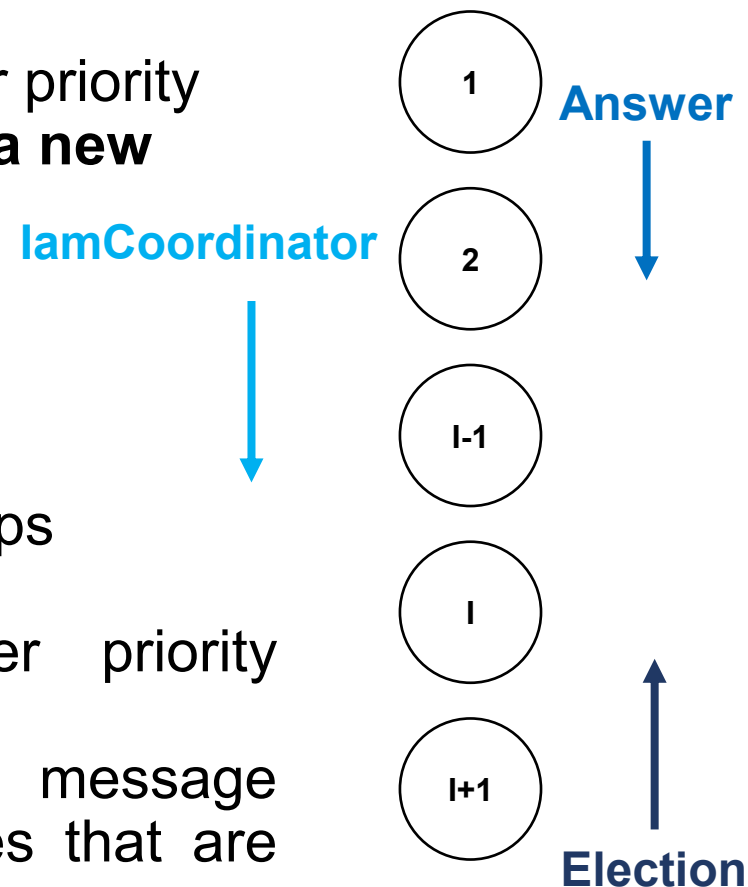
BULLY PROTOCOLS

Every **participant** can start the **election at any time**, triggered by some timeout events
sends an **election** message to **processes with higher priority (Election)**
in case of election message from a lower priority process, sends an **answer** to block and **a new election is started**

after some time, ...

Answer coordination messages from superior nodes can arrive
if they arrive, the low priority process stops

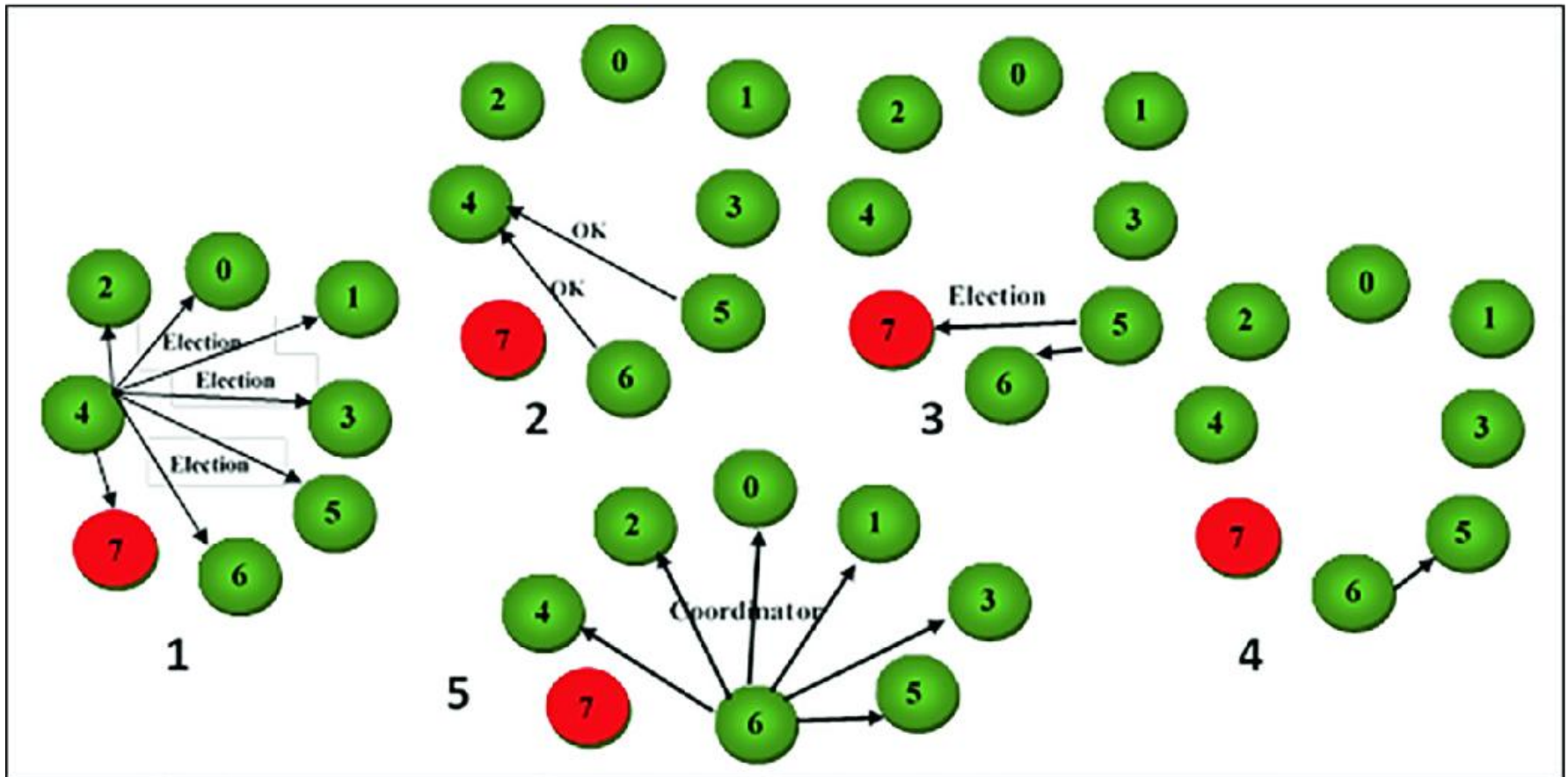
if no message arrives from higher priority processes, it becomes a coordinator and signals its presence with the message **IAmCoordinator** to lower priority nodes that are advised



BULLY PROTOCOLS

Every participant can start the **election** and several rounds can go on

Here **4** starts (not to all), blocked by 5 and 6 OK answer, then **5** goes on blocked by 6, **6** wins



GLOBAL STATE

In a distributed system it is sometimes necessary to coordinate and support a **global state associated with the current situation**

The state can be successively used to replay the system from a previous point and restart execution in a safe situation

The main point is to **locally coordinate the event of single component parts** to compose a **unique consistent view**, **without paying too much for the coordination**

- *checkpoint for recovery, distributed garbage collector*

Let us assume an **asynchronous model** with **processes** on **different nodes** that **reciprocally can send messages** (*channels are only one-way communication between processes*)

Processes can execute **locally** and **exchange messages** via **channels** that must grant that any node must reach any other one, via hops (**no partitioning**)

PROCESSES ACTIONS

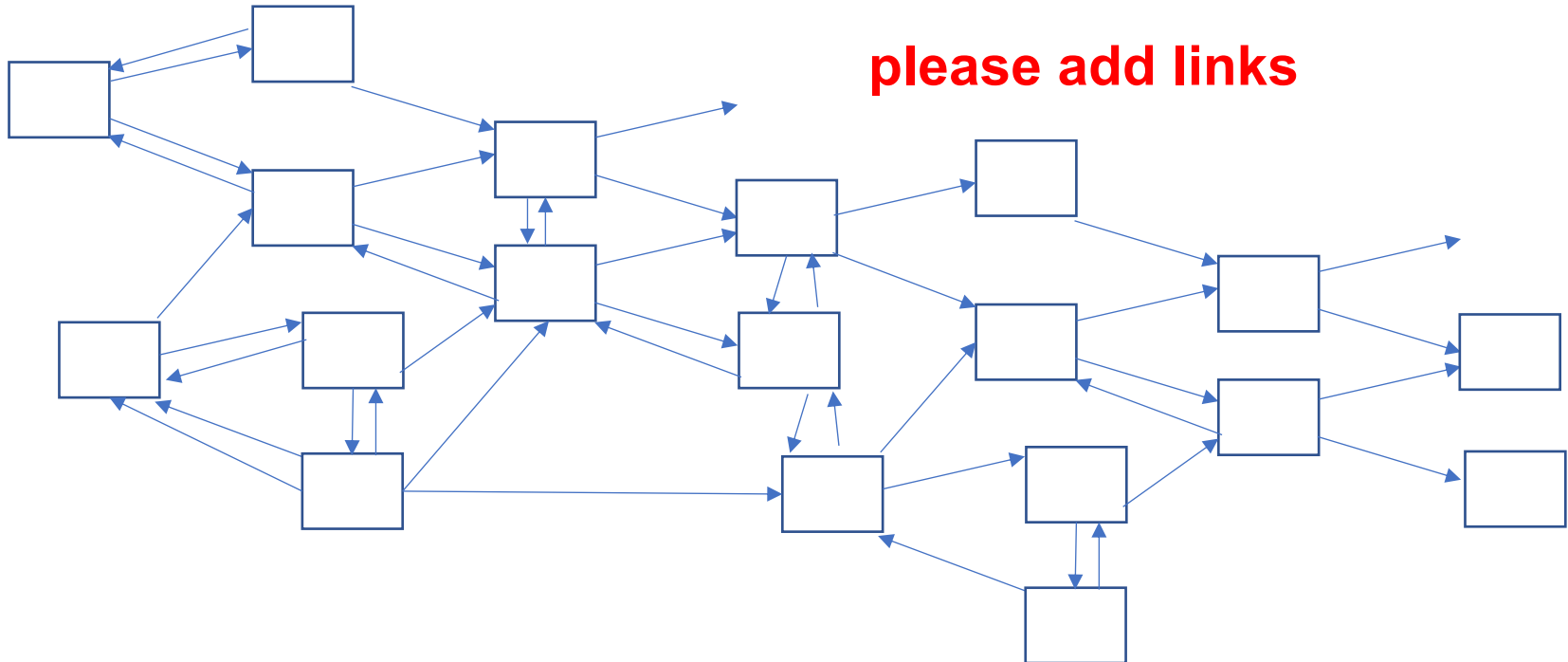
Nodes with both

In queues

Out queues



The interconnection must make possible the **reachability** of **any node from any other one (no split)** **NO PARTITIONS**



GLOBAL STATES

The **global state** stems from the **private states** of **participant processes**, but also should keep into account **exchanged messages (currently in exchange)** between different processes

The main point is to record **only the whole needed information** to avoid a situation in which you are losing any content

The snapshot must be taken while processes are running, so it must minimally intrude in the normal execution and be safe

Distributed snapshot

Compose the needed local information in a unique meaningful state but acquiring with a **distributed perspective** with a minimal coordination

Recall that we must grant a **safe global vision** in a consistent way

We have to assume a network connecting all processes with channels in such a way that there are **no partitions and any node can reach (via routing) any other node**

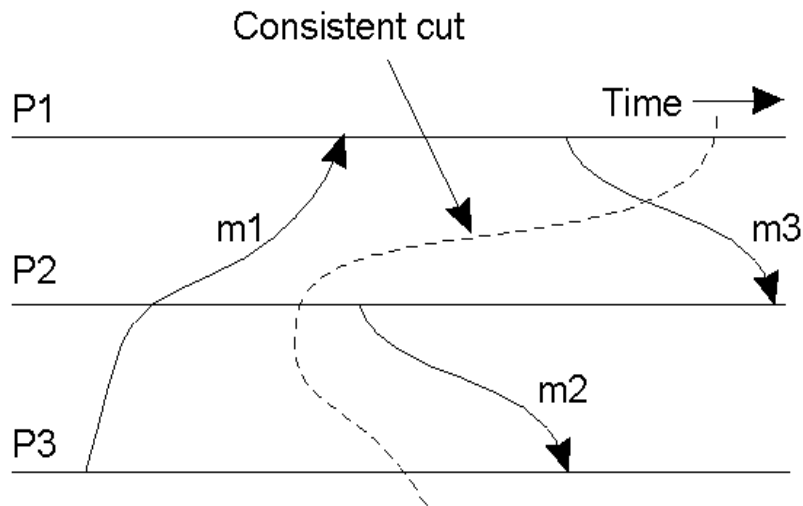
GLOBAL STATES CONSISTENCY

Consistent cuts in a distributed system (how to do it)

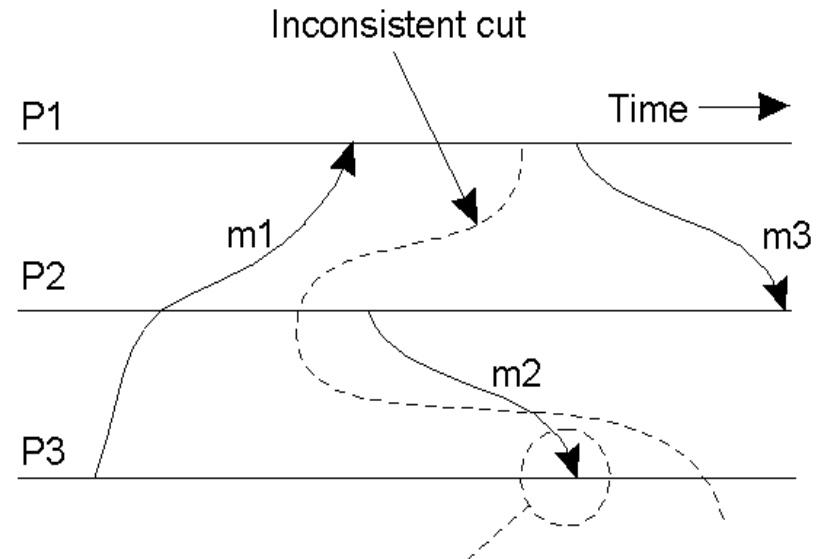
Non all states are admissible and safe for snapping the shot

Consistent cuts (a) represent a safe global state and

Inconsistent cuts (b) produce an unreasonable global state and should be avoided



(a)



Sender of m2 cannot
be identified with this cut

(b)

SAFE GLOBAL STATE

Consistent cuts in distributed system exclude unreasonable situations from the operation point of view (losing messages or duplicating them)

Consistent Cut or Message (a) message m3 from P1 to P2

In case of the m3 message, where we included the sending state in the snapping of P1, and we **must record the arrival within the state** of the receiving node P3 – input **messages must be saved**

We must keep track of that **message inside the global state** in case of replay (**these messages are part of the snapshot process, and must be recorded from the input of the receiver**)

SAFE GLOBAL STATE

Inconsistent Cut or Message (b) message m2 from P2 to P3

In case of messages where we record the arrival in the state of the receiver node, but the sending in the sender node was not recorded yet

This type of **recording or cut** is **inconsistent**, because it embeds the message in the receiver state, but the message has not been recorded in the sender state

in case of replay, the sender will forcedly resend the message that causes the effect of a **double reception** in the receiver and an **unsafe behavior** (this event must be avoided)

GLOBAL STATE VIA SNAPSHOT

Distributed Global Snapshot (one at a time)

One node starts a global snapshot

All nodes play a local algorithm to organize the local savings

The global snapshot consists of all saved states by all nodes

Any node keeps its internal state (checkpoint) and the exchanged relevant compatible messages (channels state)

Nodes do the snapshot while normally working

OBJECTIVE: to propagate a state **snapshot wave** from processes that individually record the local state; the wave expands to cover the entire system (**assumption of complete reachability**)

the global snapshot is saved on any node when the wave has propagated to all node, **so to be re-started you need a similar propagation for replay**

Where do you keep it?

How can you re-establish and restart it?

GLOBAL STATE VIA SNAPSHOT

Any node has **(more) input and (more) output channels**

One node starts the snapshot and all nodes makes the same decentralized algorithm while normally go on executing

Every process is characterized by

- **IN and OUT channels** in FIFO mode and enough connections
(*every bidirectional channel $\rightarrow$ is separated into two channels*)
- A **state** and a **color**: no snapshot, snapshot on (or over)
white - initial state (*before snapshot*)
red - successive state (*doing snapshot or completed*)
- A **marker management algorithm**: markers are messages to produce the snapshot propagation

Every **process receiving a marker or deciding a snapshot** makes a *local state save* and sends one marker message via any OUT channels

The process that receives the marker becomes **red**

The markers pass through channels in FIFO message ordering

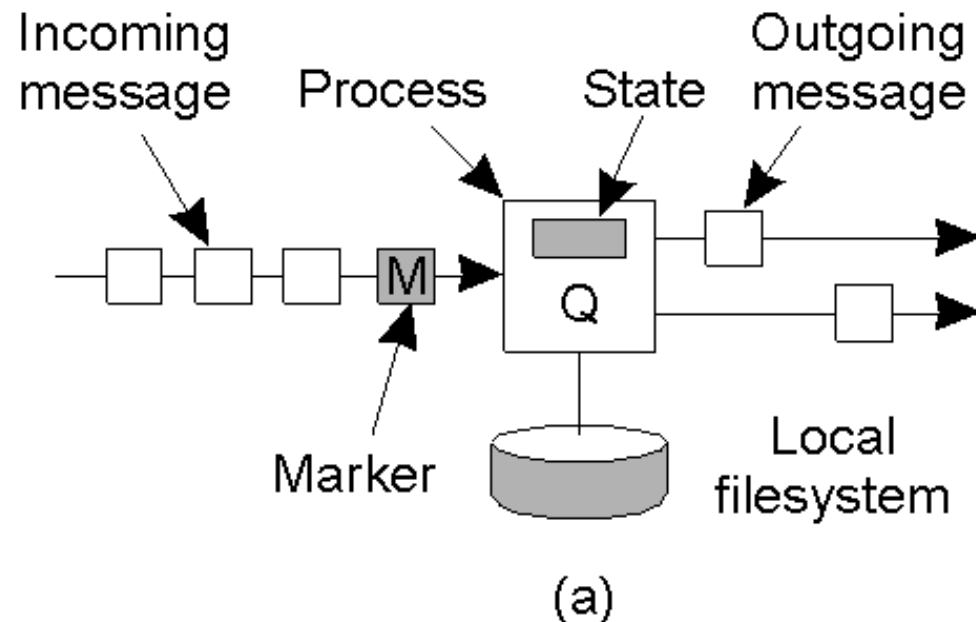
DISTRIBUTED GLOBAL SNAPSHOT

Every process organizes its state to save in two parts:

- its **local environment** (state) to record as soon as they become red
- some sets of **possible messages** associated to **input channels**
*these messages are recorded until the first **marker message** arrives in the queue*

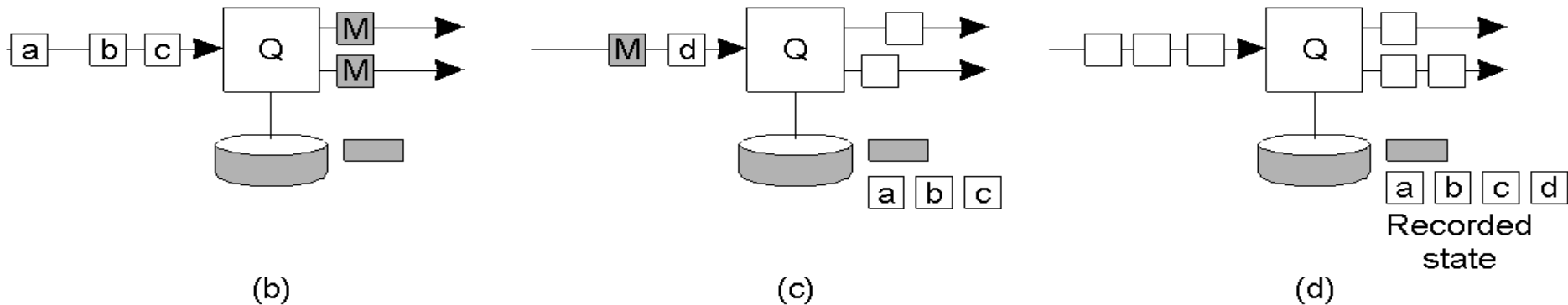
A process becomes **red** either at a **marker reception** on an input channel or if it *has decided to make a snapshot*; after that, it stays red (**stability of the state**)

A process completes its **snapshot** after receiving a marker on **every input channels** (and completes the snapshot)



- a) At the reception of M, process Q starts a snap by registering its internal state (**checkpoint**)

DISTRIBUTED GLOBAL SNAPSHOT ALGORITHM



- a) After a)
- b) The process **Q** sends out new markers to output queues and **start recording all incoming messages from open input channels**. These messages are meanwhile processed and consumed
- c) The process **Q** receives a **marker** on a specific input channel (except the one where it arrived first that is already closed)
- d) The process **Q** closes the **registration for that channel (but messages continue to be served)**

When a process ends the snapshot on all **input channels**, it has completed the **node snapshot (state plus all messages saved from input channels)**

STATE AS UNION OF LOCAL STATE

Distributed Global Snapshot

Every process can start a snapshot (checkpoint of local state) and must send the marker on every out channel

The snapshot global state result composed by:

- **local states** of every process
- **state of** input connection **channels**
(messages sent by senders and *recorded by receiver*)

For the **process state**, it is created when a process starts the snapshot or receives a marker

Every process that receives the marker makes the **checkpoint** of **its local state** e **sends a marker** message in any output queue

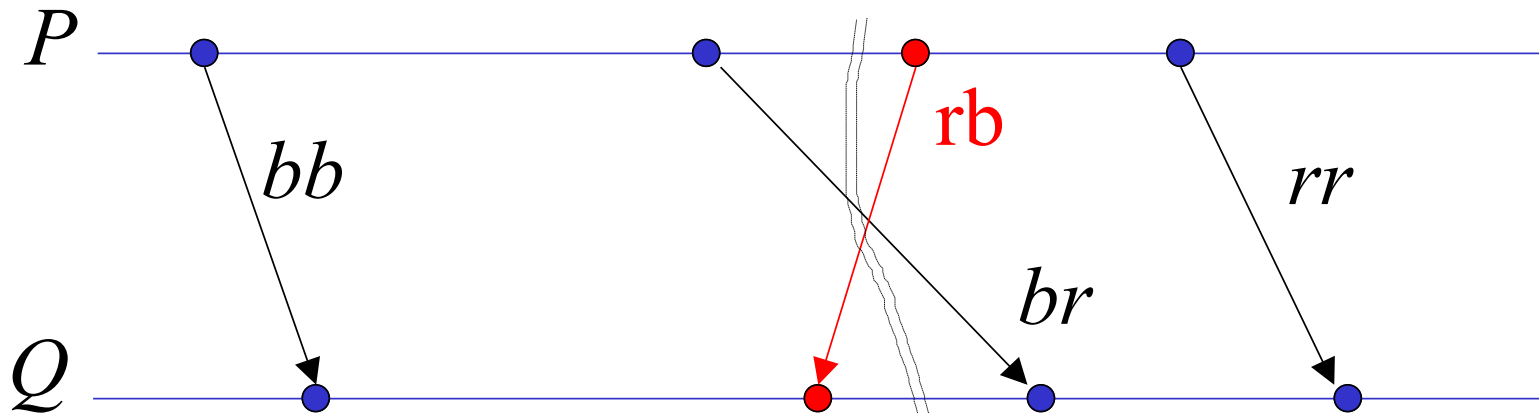
For the **channel state**, **every incoming message is recorded** until that channel gets a **marker that signals the end of the information** to be recorded for **that channel**

The registration in that channel can then be closed (*checkpoint*)

DISTRIBUTED GLOBAL SNAPSHOT

The global state is composed by:

- **local state** of every process
- **state of connection channels** (messages sent)



- **bb** messages before and **rr** messages after the snapshot
- **br** messages to be recorded in the channel state
- **rb messages not consistent** (avoided by the protocol since the marker will pass before other messages and makes the node red before the reception of other message)

Messages as **rb** are avoided by protocol construction

DISTRIBUTED GLOBAL SNAPSHOT

The process P can start a snapshot and request the collaboration of every other process that record their processor states and channel states.

How it is all recorded and where?

Every process that ends can send the state to the process that started the snapshot or to a defined node P devoted to management collection and eventual replay
(*can also keep in at its own site*)

About snapshots management

At first snapshots are intended as rare events inside the system because of the cost

- **What happen if more snapshot are executed together?**
- **How is it possible to execute more snapshots concurrently and to distinguish them?**
- **Are they compatible and how?**